

作业 1：比特币测试网交易与区块逐字节 解析

Testnet4 交易广播、SegWit 编码、区块与共识承诺验证

段沛旭 董胥宏 靳博涵

摘要

本实验在 Bitcoin Testnet4 上构造并确认一笔原生 P2WPKH 交易，保存原始序列化数据，随后以独立 Python 解析器将交易定位到每个字节和比特范围。对于包含该交易的完整区块，程序从 80 字节区块头开始消费全部 2083 字节，并独立验证区块哈希、难度目标、Merkle Root、witness commitment 与 coinbase 位置。实验强调“原始字节——字段语义——密码学承诺”三层相互校验。

关键词： Bitcoin Testnet4; SegWit; P2WPKH; Merkle Tree; Proof of Work

目录

摘要	1
1 实验目标、环境与数据集	2
2 交易序列化与字节序	2
3 交易构造、签名与广播	3
4 逐字节/逐比特解析器设计	3
5 实验交易字段详解	4
6 字段长度与交易 ID 计算	5
7 完整区块结构与交易边界	5
8 Merkle Tree 与交易承诺	6
9 nBits、目标值与 PoW	6
10 Witness Commitment 验证	7
11 节点记录与截图证据	7
12 自动校验结果与失败注入	8
13 误差来源、边界与安全说明	9
14 复现步骤、结论与参考资料	9

第 1 节 实验目标、环境与数据集

实验目标不是调用 `decoderawtransaction` 后转抄结果，而是从 raw transaction/raw block 十六进制串出发，按协议逐字段读取并记录游标。解析器输出 JSON 语义树和 TSV 字节表，使任意字段都能追溯到原始偏移。

实验对象交易 txid 为 `dea08b1b...011d5f`，序列化长度 222 bytes；所在区块长度 2083 bytes、包含 8 笔交易。节点负责产生和广播数据，独立脚本负责离线复核，避免“同一软件自证正确”。

项目	配置/结果
网络	Bitcoin Testnet4
交易类型	P2WPKH SegWit v0
交易长度	222 bytes / 1776 bits
区块长度	2083 bytes
脚本	Python 3 标准库

第 2 节 交易序列化与字节序

比特币交易由 version、输入向量、输出向量和 locktime 组成；SegWit 交易在 version 后增加 marker=00 与 flag=01，并把解锁数据从 scriptSig 移入 witness。输入引用 previous txid 时，32 字节哈希在序列化内部使用小端顺序，而人类显示时反转。

CompactSize 不是固定宽度整数：数值小于 0xfd 时单字节表示，更大数值由前缀决定后续宽度。解析器必须先读长度再读载荷，并在每一步检查剩余字节数，否则一个错误长度会使后续全部字段错位。

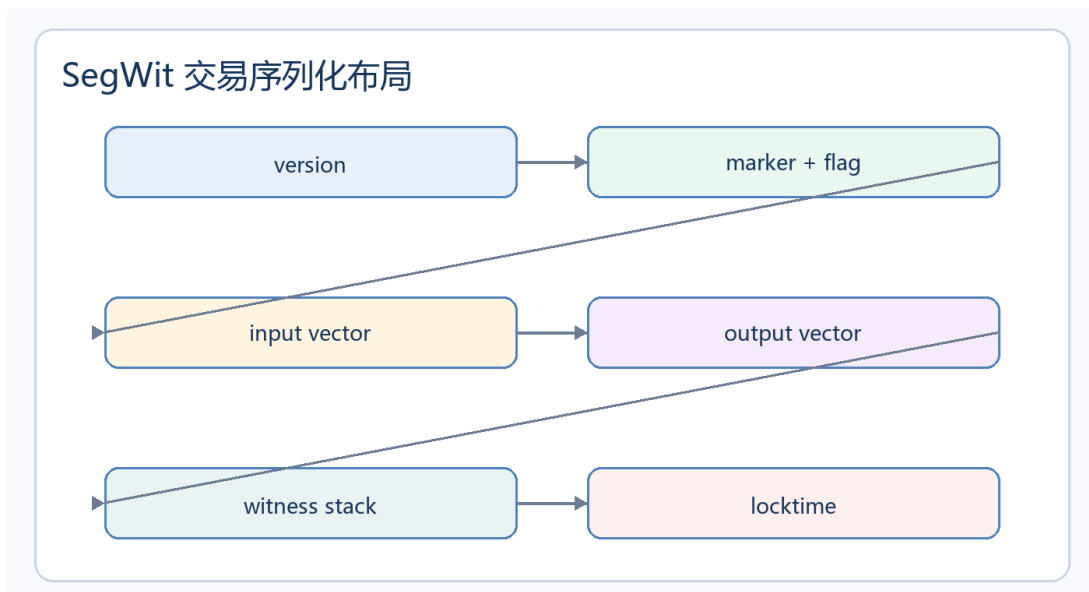


图 1: SegWit 交易的字段排列与解析顺序

第 3 节 交易构造、签名与广播

钱包从测试网 UTXO 集合选择输入，建立 10000 sat 目标输出和 323260 sat 找零输出；手续费由输入总额减去两个输出得到。原生 P2WPKH 的 scriptSig 为空，witness 栈包含 DER 编码签名与 33 字节压缩公钥。

广播前使用 `testmempoolaccept` 检查脚本、费用率、标准性与冲突；通过后调用 `sendrawtransaction`。确认阶段同时保存 wallet transaction、network transaction 与区块哈希，形成可离线复核的证据链。

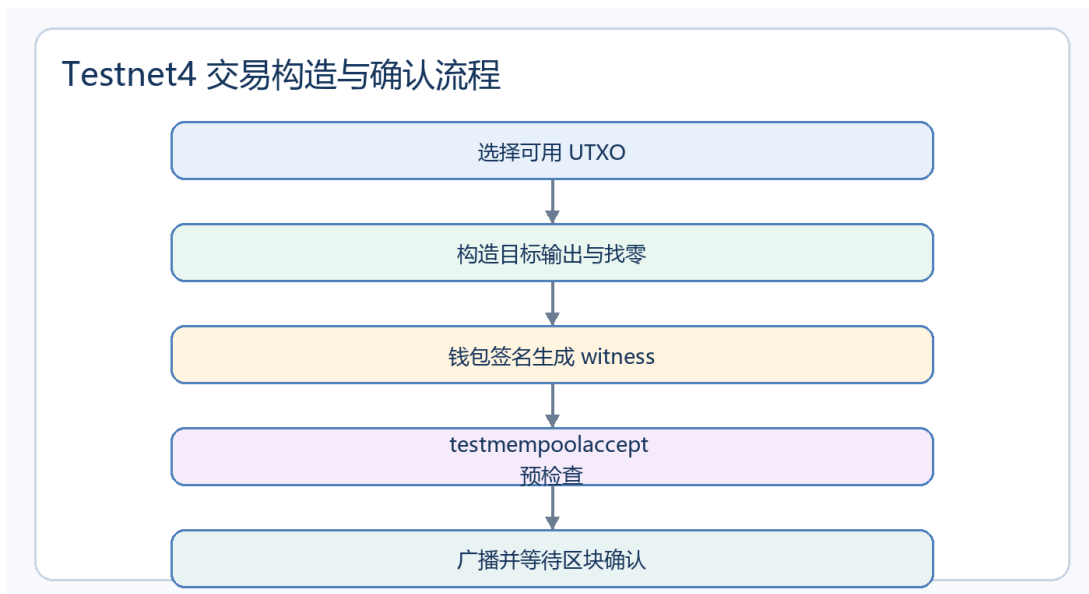


图 2: 从 UTXO 到区块确认的实验流程

第 4 节 逐字节/逐比特解析器设计

核心 Reader 保存 bytes 与 offset。`read(n, field)` 先验证 `offset+n` 不越界，再返回切片并把 offset 前移；记录器计算 `bit_start=8×byte_start` 与 `bit_end=8×(byte_end+1)-1`。这样“解析完成”不仅有字段值，还能证明没有跳过或重复消费字节。

输出行同时包含 hex、binary、decoded 三种表示。二进制按字节分组，使 0x01 显示为 00000001，而不是丢失前导零；整数解码显式指定 little-endian，哈希显示单独处理反转。

```

parse_tx.py: 游标与字段记录

1  #!/usr/bin/env python3
2  """Parse a Bitcoin transaction from raw hex without using Bitcoin Core RPC."""
3
4  import argparse
5  import hashlib
6  import json
7  from pathlib import Path
8
9
10 def sha256d(data: bytes) -> bytes:
11     return hashlib.sha256(hashlib.sha256(data).digest()).digest()
12
13
14 class Reader:
15     def __init__(self, data: bytes):
16         self.data = data
17         self.pos = 0
18         self.rows = []
19
20     def take(self, size: int, field: str, decoded=None) -> bytes:
21         start = self.pos
22         end = start + size
23         if end > len(self.data):
24             raise ValueError(f"{field}: need {size} bytes at offset {start}, only {len(self.data)-start} r

```

图 3: 解析器的游标、越界保护与字段记录实现

第 5 节 实验交易字段详解

version=2; marker/flag 表示存在 witness; 输入数为 1, previous_vout=0, sequence=0xffffffff; 输出数为 2。两个 scriptPubKey 都是 0x00 0x14 后接 20-byte key hash, 语义为 OP_0 PUSH20。

witness item 0 长 71 bytes, 末字节 0x01 表示 SIGHASH_ALL; item 1 长 33 bytes, 前缀 0x02, 表示偶 y 坐标的压缩 secp256k1 公钥。locktime=0, 因此交易不依赖高度或时间锁。

字节范围	字段	语义
0-5	version/marker/flag	v2, SegWit
6-47	input vector	1 个输入, 空 scriptSig
48-110	output vector	323260 + 10000 sat
111-217	witness	签名 + 压缩公钥
218-221	locktime	0

第 6 节 字段长度与交易 ID 计算

从字节构成看，witness 占 107/222，是最大部分；输出脚本与金额合计 63 bytes，输入非 witness 字段 41 bytes。txid 对不含 marker、flag、witness 的 legacy serialization 做 SHA256d；wtxid 则对完整 SegWit serialization 做 SHA256d，所以二者不同。

解析器重新序列化非 witness 视图并得到 txid=dea08b1b...011d5f，与节点记录一致；完整序列化得到 wtxid=df9cc534...416a29。该检查可发现字段边界正确但哈希视图选择错误的问题。

实验交易字段字节占比

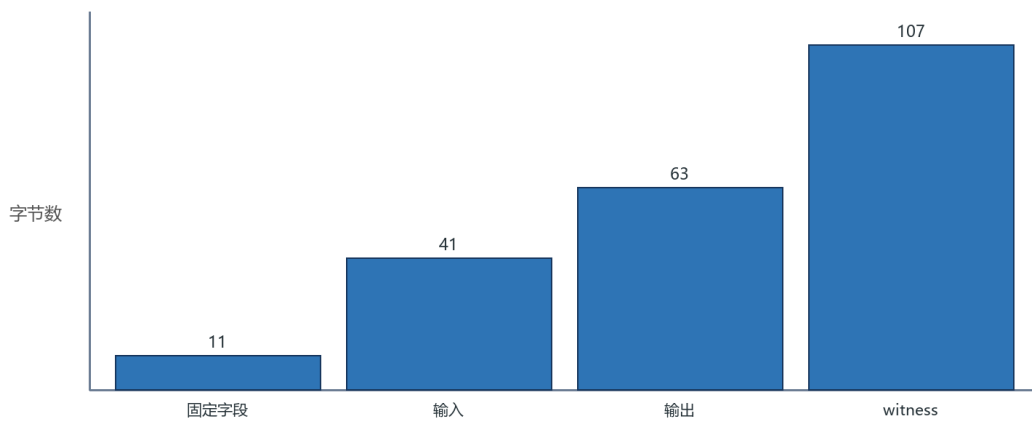


图 4: 222 字节实验交易的字段占比

第 7 节 完整区块结构与交易边界

区块以 80-byte header 开始：version、previous block hash、Merkle root、timestamp、nBits、nonce。第 81 字节为交易数量 CompactSize=8，之后 8 笔交易长度分别为 188、222、222、222、222、222、312、392 bytes。

区块解析器复用交易解析逻辑但保留区块级绝对偏移。最终 parsed_bytes 必须等于 2083，第一笔必须是 coinbase；任何尾随数据、提前结束或 coinbase 位置错误都会使验证失败。

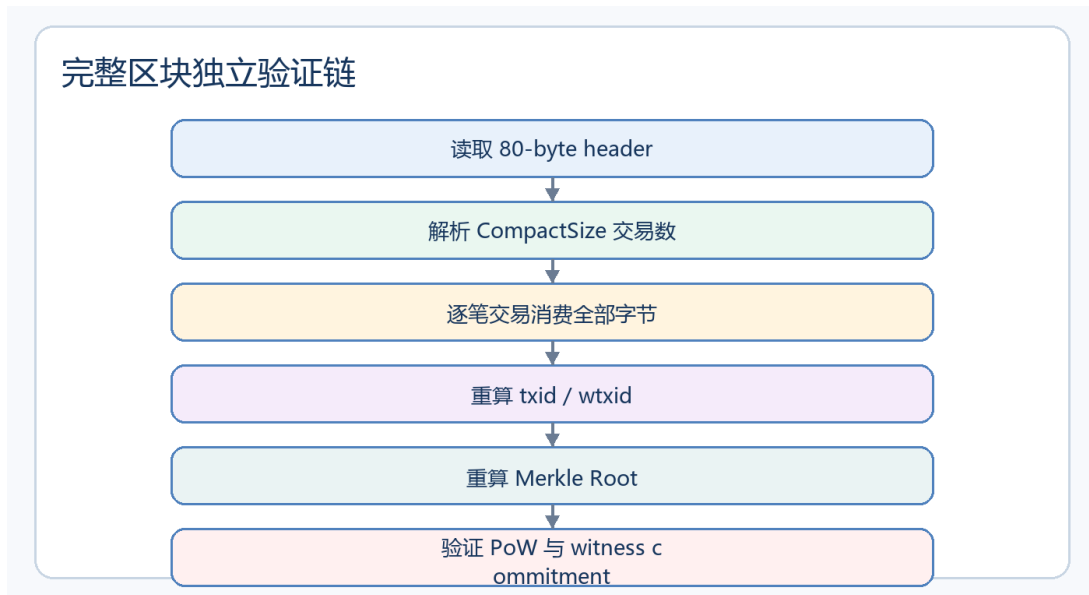


图 5: 从区块头到共识承诺的独立验证链

第 8 节 Merkle Tree 与交易承诺

Merkle tree 的叶子是按内部字节序参与拼接的 txid。每层将相邻两个 32-byte 哈希连接后 SHA256d; 若节点数为奇数, 则复制最后节点。最终根与区块头中的 af20a4e3...20c2726 完全一致。

这种树结构把任意交易修改传播到根, 再传播到区块哈希。实验不仅比较最终根, 还保存每笔交易的 byte_start、byte_end 与 txid, 从而能定位某个叶子不一致究竟来自交易解析还是树计算。

位置	长度	类型
0	188	coinbase
1-5	各 222	普通 SegWit 交易
6	312	普通交易
7	392	普通交易

第 9 节 nBits、目标值与 PoW

nBits=0x1d00ffff 是压缩目标。指数 0x1d、系数 0x00ffff, 目标 $T=0x00ffff \times 256^{(0x1d-3)}$ 。区块头 SHA256d 后按小端解释并反转显示, 所得整数必须不大于 T。

实验重算区块哈希 000000000012057b...35404bf, 小于目标 00000000ffff0000...00000000, 因此 PoW=PASS。只比较哈希字符串前导零是不充分的; 脚本使用完整 256-bit 整数比较。

```

1 coefficient = nBits & 0x007fffff
2 exponent = nBits >> 24
  
```

```
3 target = coefficient * 256 ** (exponent - 3)
4 assert int.from_bytes(hash256(header), 'little') <= target
```

第 10 节 Witness Commitment 验证

SegWit 区块还需承诺所有交易的 witness。coinbase 的 wtxid 叶子按规范置零，其他叶子使用 wtxid 构造 witness Merkle root；再与 coinbase witness reserved value 拼接并 SHA256d。

计算结果 b36e61fb...e5d0246 与 coinbase OP_RETURN 中 aa21a9ed 后的 32-byte commitment 一致。该验证说明完整 witness 字节、wtxid 计算、coinbase reserved value 与承诺脚本四者同时自洽。

项目	值/结论
witness Merkle root	b9233c8d...f20d467
reserved value	32 bytes 全零
stored commitment	b36e61fb...e5d0246
calculated commitment	完全一致, PASS

第 11 节 节点记录与截图证据

实验截图用于证明交易确实来自测试网节点流程，而非仅对随机构造的十六进制做离线演示。截图对应区块查询、交易解析和最终验证输出；提交包同时保留 JSON/TSV，截图与机器可读记录互相补充。

为了保护资金安全，报告不包含私钥、助记词、wallet.dat 或 RPC 认证信息。测试币交易公开可见，地址和交易 ID 属于本实验可复核信息。

```

"outputs": [
  {
    "n": 0,
    "value": 0.0032326,
    "value_sats": 323260,
    "script_asm": "0 4db668b024f3b4e14fbb8de2a043520b505ceea7",
    "script_hex": "00144db668b024f3b4e14fbb8de2a043520b505ceea7",
    "address": "tb1qfkmx3vpy7w6wznm3h32qs6jpdg9em484qqv7s",
    "type": "witness_v0_keyhash"
  },
  {
    "n": 1,
    "value": 0.0001,
    "value_sats": 10000,
    "script_asm": "0 be78ccbbd06737973a759b99cf5e3314c65c6c90",
    "script_hex": "0014be78ccbbd06737973a759b99cf5e3314c65c6c90",
    "address": "tb1qheuvew7svumewwn4nvvu7h3nzn9cmysnhhs3",
    "type": "witness_v0_keyhash"
  }
]

```

图 6: Testnet4 区块/交易实验过程截图

第 12 节 自动校验结果与失败注入

最终七项检查全部为 PASS: all bytes consumed、transaction count、block hash、PoW、Merkle Root、witness commitment、coinbase first。检查之间具有独立性，例如字节消费通过不代表哈希或承诺正确。

为验证检测能力，可在副本中改变一个输出金额字节：解析仍可能完成，但 txid、Merkle root、block hash 会改变；若只改变 witness 签名，则 txid 不变而 wtxid/witness commitment 改变。这些差分体现 SegWit 设计目标。

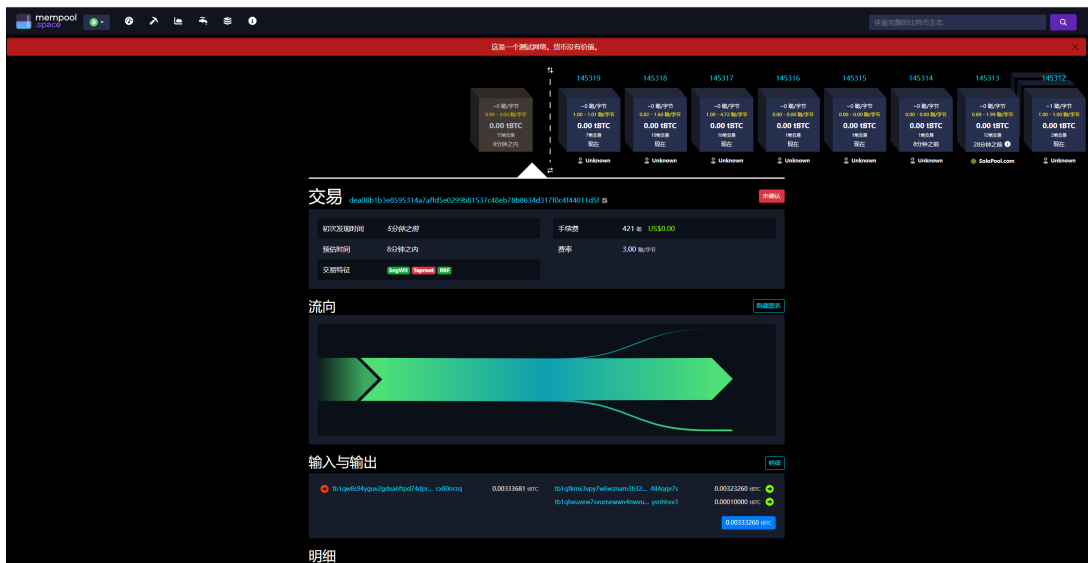


图 7: 独立验证脚本的最终 PASS 结果

第 13 节 误差来源、边界与安全说明

常见错误包括哈希字节序反转两次、把 CompactSize 当固定 4 bytes、计算 txid 时错误保留 witness、Merkle 奇数层未复制末项、nBits 只按前导零近似判断。报告中的每项校验都针对一种错误类别。

本实验只在 Testnet4 操作，脚本是协议解析器而不是共识实现，未覆盖全部脚本类型、Taproot、非标准交易与恶意资源消耗。生产系统应使用经过审计的 Bitcoin Core，并对输入长度设置资源上限。

- 只使用测试币，绝不把测试私钥迁移到主网。
- raw block 来自可信节点；若来自网络对端，还需限制最大消息长度。
- TSV 的逐 bit 展示用于教学，不应用作高性能网络解析。

第 14 节 复现步骤、结论与参考资料

在无节点环境下也可使用提交包中的 05-signed.hex 与 10-block.hex 完成全部离线校验。运行后应得到 222/222、2083/2083 以及所有 PASS；输出 JSON 可与随附基准文件做结构化比较。

实验完成了从真实交易广播到协议字节、密码学哈希和区块共识承诺的闭环。逐字节边界、txid/wtxid、Merkle/PoW/witness commitment 相互印证，满足“解析到每一个 bit/byte”的要求。

```
1 python code/parse_tx.py data/05-signed.hex --expected-txid dea08b1b
   ... --prefix out/tx
2 python code/parse_block.py data/10-block.hex --expected-hash
   000000000012057b... --prefix out/block
```